



Lisp

Tears of Joy

Part—5

Lisp has been hailed as the world's most powerful programming language. But only the top percentile of programmers use it, because of its cryptic syntax and academic reputation. This is rather unfortunate, since LISP isn't that hard to grasp. If you want to be among the *crème de la crème*, this series is for you. This is the fifth article in the series that began in the June 2011 issue of *LFY*.

Lisp is not slow, courtesy macros

Doug Hoyte, creator of Antiweb (hoytech.com/antiweb), gives a lot of credit to macros for efficient Lisp performance. He says that while other languages give you small, square tiles, Lisp lets you pick tiles of any size and of any shape. With C, programmers use a language that is directly tied to the capabilities of a fancy *fixnum* adder. Aside from procedures and structures, little abstraction is possible in C. By contrast, Lisp was not designed around the capabilities and limitations of the machines.

Instead of enquiring what makes a program fast, it's better to ask what makes a program slow. The root causes can be roughly classified into three broad categories: bad algorithms, bad data structures, and general code. All language implementations need good algorithms. An algorithm is a presumably well-

“If you tile a floor with tiles the size of your thumbnail, you don't waste many.”

—Paul Graham

researched procedural description of how to execute a programming task. Because the investment required in coming up with an algorithm is so much larger than that of implementing one, the use of algorithms is ubiquitous throughout all of computer science. Somebody